

Comparing (Spring MVC + Virtual Threads) with Spring WebFlux

The Same Problem, Two Different Philosophies

- **Spring MVC** with virtual threads says: blocking I/O is fine, just make threads cheap enough that blocking thousands of them simultaneously costs almost nothing.
- **Spring WebFlux** says: never block at all, use a tiny fixed thread pool and handle thousands of concurrent operations through a non-blocking event loop and reactive pipelines.

Both philosophies work. The question is which one fits your specific situation better.

Coding Style

Before any performance or scalability discussion, this is the one that affects your team every single day. Take the same `placeOrder` business logic and read both versions carefully.

Spring MVC version

```
public OrderResponseDto placeOrder(OrderRequestDto request) {  
  
    Product product = productService.findById(request.getProductId())  
        .orElseThrow(() -> new ProductNotFoundException("Product not found"));  
  
    if (product.getStock() < request.getQuantity()) {  
        throw new InsufficientStockException("Only " + product.getStock() + " items available");  
    }  
}
```

```

    }

    User user = userService.findById(request.getUserId())
        .orElseThrow(() -> new UserNotFoundException("User not found"));

    BigDecimal finalPrice = (user.isPremium() && request.getQuantity() >= 5)
        ? applyPremiumDiscount(product, request.getQuantity())
        : product.getPrice().multiply(BigDecimal.valueOf(request.getQuantity()));

    if (!paymentService.checkBalance(user.getId(), finalPrice)) {
        throw new InsufficientBalanceException("Insufficient balance");
    }

    Order savedOrder = orderRepository.save(createOrderEntity(product, user, request, finalPrice));
    productService.reduceStock(product.getId(), request.getQuantity());
    return convertToResponseDto(savedOrder, user, product);
}

```

Spring WebFlux version

```

public Mono<OrderResponseDto> placeOrder(OrderRequestDto request) {
    return Mono.zip(findProduct(request), findUser(request))
        .flatMap(tuple -> processOrder(tuple.getT1(), tuple.getT2(), request))
        .onErrorMap(this::mapToBusinessException)
        .doOnError(ex -> log.error("Order placement failed: {}", ex.getMessage()));
}

```

```

}

private Mono<OrderResponseDto> processOrder(Product product,
User user, OrderRequestDto request) {
    if (product.getStock() < request.getQuantity()) {
        return Mono.error(new InsufficientStockException(
            "Only " + product.getStock() + " items availabl
e"));
    }
    return calculateFinalPrice(product, user, request)
        .flatMap(finalPrice -> validateBalanceAndCreateOrder(
            product, user, request, finalPrice));
}

private Mono<OrderResponseDto> validateBalanceAndCreateOrder(
    Product product, User user, OrderRequestDto request,
    BigDecimal finalPrice) {
    return paymentService.checkBalance(user.getId(), finalPrice)
        .flatMap(hasBalance -> {
            if (!hasBalance) return Mono.error(new InsufficientBalanceException("Insufficient balance"));
            return saveOrderAndReduceStock(product, user, request, finalPrice);
        });
}

private Mono<OrderResponseDto> saveOrderAndReduceStock(
    Product product, User user, OrderRequestDto request,
    BigDecimal finalPrice) {
    return orderRepository.save(createOrderEntity(product, user, request, finalPrice))
        .flatMap(savedOrder -> productService
            .reduceStock(product.getId(), request.getQuantity()))
        .thenReturn(savedOrder)
}

```

```

        .map(savedOrder -> convertToResponseDto(savedOrder, user, product));
    }

```

Both produce identical results. But notice how much more scaffolding the reactive version requires to express the same business logic.

Aspect	Spring MVC + Virtual Threads	Spring WebFlux
Readability	Linear, sequential, reads top to bottom	Pipeline-based, requires thinking in operators
Cognitive load	Low, familiar Java control flow	High, new mental model required
Error handling	<code>try-catch</code> and <code>orElseThrow</code>	<code>onErrorMap</code> , <code>onErrorResume</code> , <code>switchIfEmpty</code>
Stack traces	Complete, readable	Fragmented across operator boundaries
Debugging	Standard breakpoints work normally	Harder, reactor-specific tooling needed
Validation	<code>@Valid</code> works automatically everywhere	Manual <code>Validator</code> injection required
Database	JPA, full relationship support, <code>ddl-auto</code>	R2DBC, no relationships, manual SQL only
Testing	Standard <code>@SpringBootTest</code> , JUnit assertions	Requires <code>StepVerifier</code> for reactive assertions
Migration cost from existing MVC app	One config line, zero code changes	Full rewrite of every service and repository
Team onboarding	Any Java developer is productive immediately	Reactive programming learning curve: weeks to months
Streaming support	<code>SseEmitter</code> , works but no backpressure	Native, first-class, backpressure built in
Concurrency model	One VT per request, JVM manages parking	One event loop, never blocks

Debugging

Virtual threads stack trace:

```
java.lang.RuntimeException: User not found
    at com.example.UserService.findById(UserService.java:34)
    at com.example.OrderService.placeOrder(OrderService.java:
22)
    at com.example.OrderController.createOrder(OrderController.java:41)
    at ...Tomcat thread handling
```

WebFlux stack trace:

```
java.lang.RuntimeException: User not found
    at com.example.UserService.lambda$findById$0(UserService.
java:34)
    at reactor.core.publisher.Mono.lambda$flatMap$0(Mono.jav
a:... )
    at reactor.core.publisher.FluxFlatMap$FlatMapMain.onNext
(...)
    at reactor.core.publisher.MonoZip$ZipCoordinator.signal
(...)
    at reactor.core.publisher.FluxSubscribeOnCallable$Callabl
eSubscribeOnSubscription.run(...)
    at reactor.core.scheduler.ExecutorSchedulerWorker.lambda
$schedule$0(...)
    ... 14 more reactor internal frames
    at com.example.OrderService.placeOrder(OrderService.java:
18) -- your code is somewhere in here
```

The reactive stack trace buries your actual code in dozens of reactor internal frames. Finding the real line that threw the error requires experience with reading reactive stack traces, which is a learned skill in itself.

SSE Comparison

WebFlux SSE

```
@GetMapping(value = "/users/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<UserResponse> streamUsers() {
    return userService.findAll(); // backpressure automatic,
    consumer controls pace
}
```

Spring MVC SSE with Virtual Threads

```
@GetMapping(value = "/users/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public SseEmitter streamUsers() {
    SseEmitter emitter = new SseEmitter(Long.MAX_VALUE);

    Thread.startVirtualThread(() -> {
        try {
            List<UserResponse> users = userService.findAll();
            // blocking JDBC, fine on VT
            for (UserResponse user : users) {
                emitter.send(SseEmitter.event()
                    .id(String.valueOf(user.getId()))
                    .name("user")
                    .data(user));
                Thread.sleep(50); // manual throttle -- no backpressure here
            }
            emitter.complete();
        } catch (Exception e) {
            emitter.completeWithError(e);
        }
    });
}
```

```
return emitter;  
}
```



The `Thread.sleep(50)` in the MVC version throttles how fast you send to the client, but the database has already returned all rows into memory before the loop even starts. With WebFlux and R2DBC, the database itself is throttled, rows are fetched in small batches as the downstream subscriber requests them, so memory usage stays flat regardless of result set size.

	WebFlux SSE	Spring MVC SSE + VT
Backpressure	Automatic	None, manual throttle
Code complexity	Low, return <code>Flux</code> with <code>TEXT_EVENT_STREAM</code>	<code>SseEmitter</code> with manual thread management
Memory at 10k connections	Very low, event loop based	Low, one parked VT per connection
Memory at 100k connections	Still very low	Starting to matter

Where Virtual Threads Win Clearly

1. Standard CRUD applications

The overwhelming majority of business applications are request-response CRUD, fetch some data, apply some business rules, save something, return a result. For this workload, virtual threads handle massive concurrency with zero code change. There is no compelling reason to adopt the reactive programming model.

2. Team productivity and maintainability

A Spring MVC codebase with virtual threads can be understood, debugged, and extended by any Java developer hired off the street. A WebFlux codebase requires reactive programming knowledge that takes weeks to build and produces bugs that are genuinely difficult to diagnose.

3. Relational data with complex joins

JPA with `@OneToMany`, `@ManyToOne`, lazy loading, and `ddl-auto` is decades of mature tooling. R2DBC has none of this. For any application with meaningful relational data, staying on JDBC and JPA with virtual threads is the path of least resistance.

4. Migrating existing applications

```
# entire migration from blocking Spring MVC to virtual threads
spring:
  threads:
    virtual:
      enabled: true
```

That is literally it. Zero code changes. An existing MVC application gets the concurrency benefits of virtual threads with one line of configuration. The equivalent WebFlux migration requires rewriting every controller, service, and repository in the application.

Where WebFlux Wins Clearly

1. Backpressure

This is the one scenario where WebFlux has a genuine architectural advantage that virtual threads cannot replicate without significant manual work.

```
// WebFlux -- backpressure is automatic and built into the
Flux contract
Flux<User> stream = userRepository.findAll(); // DB produces rows as subscriber requests them
stream
    .buffer(100) // batch into groups of 100
    .delayElements(Duration.ofMillis(50)) // subscriber controls pace
    .subscribe(batch -> process(batch));
```

```
// Virtual threads -- no backpressure, all rows loaded into memory
List<User> all = userRepository.findAll(); // entire result set in JVM memory at once
```

If your system has a fast producer and a slow consumer, a database emitting rows faster than a client can receive them for example, WebFlux's backpressure prevents memory overflow automatically. Virtual threads have no equivalent mechanism.

2. Streaming at extreme scale

100,000 concurrent SSE connections:

- With Virtual Threads: 100,000 VTs parked in heap memory, each holding a stack frame, consuming memory per connection.
- With WebFlux: ~8 event loop threads all 100,000 connections multiplexed through the Netty event loop memory cost per connection is negligible

At moderate scale, a few thousand connections, the difference is not meaningful. At extreme scale, 50,000 to 100,000 concurrent long-lived connections, WebFlux's memory footprint is significantly lower.

3. Composing multiple event streams

Operators like `merge`, `zip`, `debounce`, and `buffer` across multiple streams have no virtual thread equivalent you can reach for. You would need to implement them manually. If your system genuinely processes and composes multiple real-time data streams simultaneously, the reactive operator vocabulary is a serious productivity advantage.

```
// combining two live streams into one -- trivial in WebFlux
Flux<Event> combined = Flux.merge(kafkaStream, websocketStream)
    .filter(event -> event.isImportant())
    .bufferTimeout(100, Duration.ofSeconds(1))
    .flatMap(batch -> processBatch(batch));
```

4. Reactive-only drivers

R2DBC, reactive Kafka (`reactor-kafka`), reactive Redis (`lettuce`), reactive MongoDB. These libraries are built on the reactive streams specification and have no blocking adapter. If your technology stack includes any of these, you are already in the reactive world and WebFlux is the natural fit.

The hybrid approach

Run Spring MVC + VT for your main application. Use WebFlux only for the specific service that needs streaming or extreme concurrency. A dedicated notification or streaming service in WebFlux sits alongside a conventional MVC backend. Each technology where it fits, not applied uniformly across everything.

Bottom Line

Before Java 21, teams adopted WebFlux just to handle concurrency because a 200-thread Tomcat pool scaled poorly. That reason no longer exists. Now the decision can be made:

- Need backpressure, streaming, or extreme connection counts? Use **WebFlux**.
- Everything else? **Spring MVC** + Virtual Threads, better readability, better debugging, zero migration cost, competitive scalability for most real workloads.

 The best engineers now are not the ones who always choose reactive or always choose virtual threads. They are the ones who understand both well enough to choose deliberately and resist the temptation to over-engineer for scale they do not actually have yet.